# ECEN-4243 Lab 3: Pipelined RISC-V Microarchitecture

Landon Schultz (CWID: A20416726)
Kiaan Doolabh (CWID: A20429908)

April 27, 2026

# 1 Introduction

The goal of this lab was to take the single cycle RISC-V processor from the previous lab and extend it into a pipelined processor. This required us to break the datapath into pipeline stages, add pipeline registers between those stages, and then handle the new hazards that appear once multiple instructions are executing at the same time. The main focus of the lab was not just getting the processor to run, but also verifying that it behaved correctly in simulation and then comparing the pipelined version against the single cycle version using synthesis results.

By the end of this lab, we had completed the pipelined processor design, verified it in ModelSim using the provided test programs, and gathered timing and area results for both the single cycle and pipelined processors using Synopsys Design Compiler. The FPGA implementation portion of the lab became bonus and was not completed, so this report focuses on the completed work from the introduction, pipelined design and testing, and the ASIC synthesis comparison.

# 2 Design

## 2.1 System Overview

Our Lab 3 processor followed the classic five stage pipelined RISC-V structure:

- Instruction Fetch (IF)
- Instruction Decode / Register Read (ID)
- Execute (EX)
- Memory Access (MEM)
- Writeback (WB)

Compared to the Lab 2 single cycle design, the main difference is that several instructions can now be active in different stages of the processor at the same time. This improves performance, but it also adds much more control complexity because the processor has to correctly deal with hazards, forwarding, and control flow changes while still preserving correct instruction behavior.

## 2.2 Pipeline Structure

To support this structure, we added pipeline registers between the major stages of the datapath. These registers hold both the data values and the control values needed by later stages. The design uses a Fetch to Decode register, a Decode to Execute register, an Execute to Memory register, and a Memory to Writeback register to move information cleanly between stages.

This allowed each stage to work on a different instruction each cycle once the pipeline was filled. The general datapath still performs the same jobs as before, such as register reads, ALU operations, memory access, and register writeback, but now these operations are separated across multiple cycles and multiple stages.

## 2.3 Instruction Support

The pipelined processor was built on top of the instruction support from Lab 2, so it still had to correctly execute the RV32I instructions we had already implemented and verified. This included:

Arithmetic instructions such as `add` and `sub` were still supported in the pipelined version, along with logical instructions such as `and`, `or`, and `xor`. Comparison operations such as `slt` and immediate instructions such as `addi` were also preserved from the Lab 2 design.

The memory side of the processor still had to support instructions such as `lw`, `lb`, `lh`, `sw`, `sb`, and `sh`. In addition to this, branch instructions such as `beq`, `bne`, and `blt`, along with jump instructions such as `jal` and `jalr`, all had to continue working correctly once the datapath was pipelined.

This part of the lab was not about adding a completely new ISA, but instead making sure that the existing instruction support from Lab 2 still worked correctly once pipelining was introduced.

### 2.4 Hazard Handling

The biggest design challenge in this lab was hazard handling. In the single cycle processor, we did not need to worry about one instruction interfering with another because only one instruction was actively moving through the datapath each cycle. In the pipelined design, this was no longer true.

The hazards we had to account for were data hazards, where one instruction depends on a result that has not yet been written back, load-use hazards, where a value being loaded from memory is needed immediately by the following instruction, and control hazards, where a branch or jump changes the PC after later instructions have already been fetched.

To solve these problems, we used a combination of forwarding, stalling, and flushing. Forwarding allowed values from later stages to be routed back into the execute stage without waiting for writeback. Stalling was used when forwarding alone was not enough, especially for load-use cases. Flushing was used to remove incorrect instructions after branches or jumps redirected control flow.

### 2.5 Control and Datapath Changes

The controller had to be updated so that control signals were generated at the correct stage and passed through the pipeline registers with the corresponding instruction. This meant that signals such as register write enable, result source, memory write enable, and ALU control had to be stored and moved through the pipeline instead of being used all at once in a single cycle.

The datapath was also updated to support forwarding multiplexers, hazard control logic, and the stage registers themselves. Even though the main functional blocks such as the ALU, register file, immediate extension, and memory logic were still familiar from Lab 2, the way they interacted had to be changed significantly to support the pipelined execution flow.

## 3 Implementation

### 3.1 System Verilog Work

The implementation was completed inside the pipelined RISC-V source file, where the partially completed template was updated into a working processor. Most of the work involved finishing the datapath and controller connections, making sure the pipeline registers were storing the correct values, and adding the hazard support needed for correct execution.

The major areas that had to be completed or debugged included the pipeline stage register connections, the forwarding path logic, the hazard detection and stall logic, the flush control for branches and jumps, the memory stage and writeback stage signal flow, and the top level simulation support for switching test files.

### 3.2 Representative Code Features

One of the most important parts of the design was the hazard unit, since this is what allowed the pipelined processor to preserve correctness even when instructions overlapped. The hazard unit decides when values can be forwarded, when a load-use dependency requires a stall, and when the fetch and decode stages must be flushed because of a control transfer.

Another major feature was the use of pipeline registers to carry both data and control information from one stage to the next. This is what keeps the processor organized, because each instruction must bring its needed control signals along with it as it moves through the stages.

Finally, we had to make sure the testbench and memory setup were flexible enough to allow us to run the professor provided tests one at a time by selecting a memory file. This made simulation much easier, since we could change the test input without rewriting the rest of the design.

## 4 Testing Strategy

Our testing for this lab was done in ModelSim. Since pipelining adds much more complexity than the single cycle design, we did not want to rely on only one passing test. Instead, we used multiple professor provided tests to make sure the processor behaved correctly across arithmetic, branching, memory access, and jump behavior.

### 4.1 Simulation Approach

For each test, we loaded the corresponding memfile, ran the simulation, and then inspected the register and memory behavior through the waveform window. We focused especially on whether values appeared in the expected registers at the correct points in time, whether branch and jump redirections looked correct, and whether memory writes matched the expected outputs described by the test program.

Some of the main tests we used included:

- `riscvtest`
- `pipe-test`
- `test_hw2`

These tests were useful because together they exercised both the basic instruction functionality and the pipelining behavior that the lab was trying to teach.

### 4.2 Waveform Verification

For this lab, the waveform output was especially important. Instead of only checking that the simulation stopped in the right place, we also used the waveform to verify that the register values and memory operations matched the expected values from the provided tests. This gave us better confidence that the processor was not only finishing, but actually operating correctly internally.

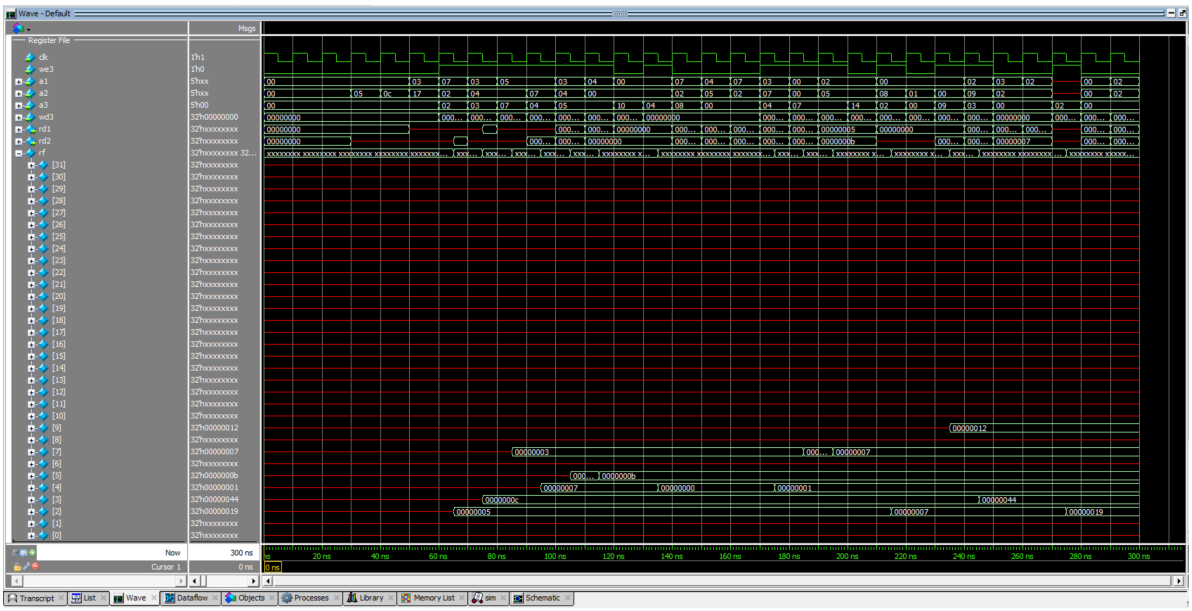

Figure 1: Representative ModelSim waveform for the riscvtest program.

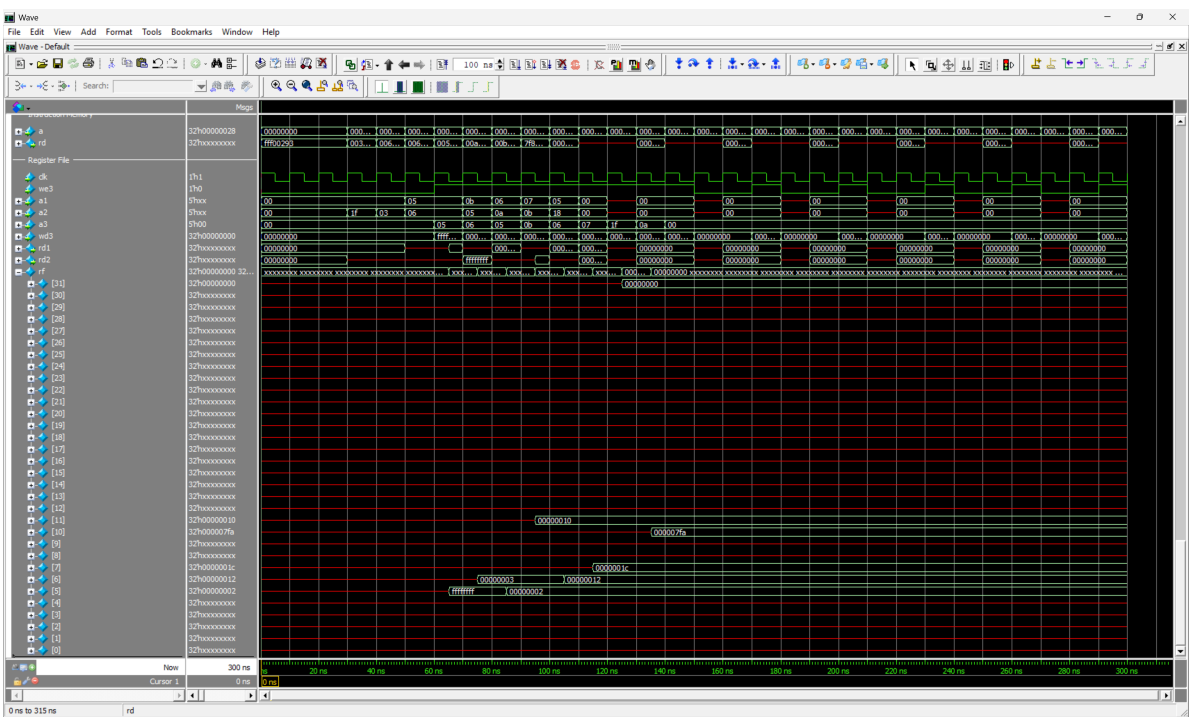

Figure 2: Representative ModelSim waveform for the pipe-test program.

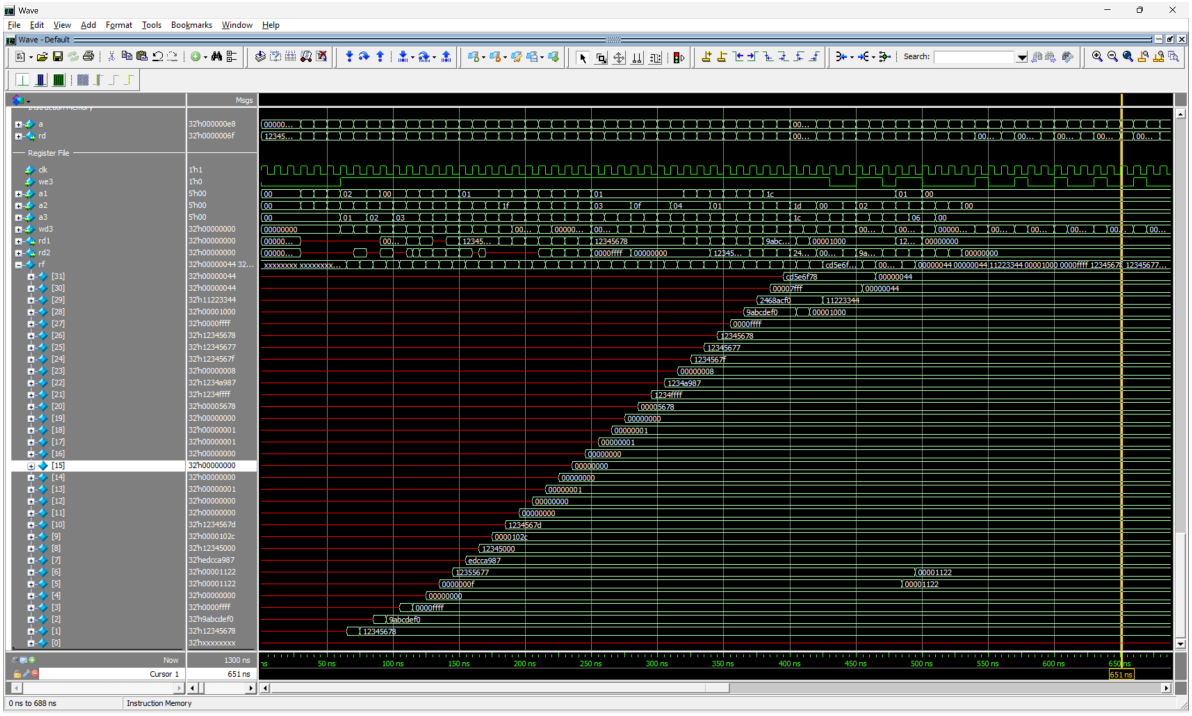

Figure 3: Representative ModelSim waveform for the test_hw2 program.

### 4.3 Testing Discussion

The most useful part of the testing process was being able to compare the waveform activity directly against the expected instruction behavior from the test files. This made it much easier to judge whether the pipeline was handling hazards correctly. A design can sometimes appear to run successfully while still forwarding or stalling incorrectly, so using the waveform as a manual verification tool was very important in this lab.

## 5 Simulation Results

### 5.1 General Results

After going through the professor provided tests, the processor was able to run the selected programs in ModelSim and produce waveform behavior consistent with the expected instruction results. In particular, the observed register and memory values matched what the tests described for arithmetic results, branch decisions, jump behavior, and final stored outputs.

### 5.2 Pipeline Behavior

One of the main things we looked for in the waveforms was evidence that the pipeline was actually behaving like a pipeline rather than just a slower single cycle machine. We were able to see instructions progressing through the different stages, along with the effects of forwarding and hazard control when dependent instructions appeared close together.

This was especially important in the provided pipelined tests, where the design had to avoid incorrect values caused by overlapping instructions. The waveforms showed that the processor was stalling and forwarding where needed rather than simply allowing bad data to pass through.

# 6 ASIC Synthesis and Comparison

## 6.1 Synthesis Method

For the final part of the lab, we synthesized both the single cycle processor from Lab 2 and the pipelined processor from Lab 3 using Synopsys Design Compiler. The purpose of this comparison was to look at the tradeoff between hardware cost and performance. Since pipelining adds registers, hazard logic, and more control complexity, we expected the pipelined design to use more area. However, we also expected it to support a higher clock frequency.

We used the server flow provided in the lab and collected timing and area reports at both a low frequency and a higher frequency for each processor. For the single cycle processor, the results we used in our report came from 10 MHz, 180 MHz, 190 MHz, and 200 MHz testing. For the pipelined processor, the summary results we used were from 10 MHz and 300 MHz.

## 6.2 Single-Cycle Synthesis Summary

The single cycle processor synthesized successfully and met timing at the lower frequency point easily. It also continued meeting timing at higher frequencies up through 200 MHz, although the slack became much smaller as the clock period decreased. This matched what we would expect for a single cycle design, since the entire instruction path must fit inside one clock cycle.

## 6.3 Pipelined Synthesis Summary

The pipelined processor also synthesized successfully. As expected, it used more area because of the added pipeline registers and hazard handling logic. However, it was able to operate at a much higher maximum frequency than the single cycle processor. This is one of the main advantages of pipelining, since the work of an instruction is divided across multiple shorter stages.

## 6.4 Area Comparison

Table 1: Area comparison between the single cycle and pipelined processors.

| Metric | Single-Cycle | Pipelined |
|---|---|---|
| Total cell area | 33,964.84 $\mu m^2$ | 56,051.10 $\mu m^2$ |
| Combinational area | 20,856.36 $\mu m^2$ | 22,910.44 $\mu m^2$ |
| Non-combinational area | 13,108.48 $\mu m^2$ | 33,140.66 $\mu m^2$ |

These results make sense, since the pipelined processor contains many more registers and control paths than the single cycle version.

### 6.5 Timing Comparison

Table 2: Timing comparison between the single cycle and pipelined processors.

| Metric | Single-Cycle | Pipelined |
|---|---|---|
| Low frequency tested | 10 MHz | 10 MHz |
| High frequency tested | 200 MHz | 300 MHz |
| Critical path at high frequency | 4.851 ns | 3.267 ns |
| Slack at high frequency | +0.103 ns | +0.005 ns |

This comparison shows the tradeoff very clearly. The single cycle processor is smaller, but its full instruction path limits the clock speed. The pipelined processor is larger, but it supports a higher operating frequency because the work is spread out across several stages.

### 6.6 Comparison Table

Table 3: Summary comparison of Lab 2 and Lab 3 processors.

| Category | Single-Cycle Processor | Pipelined Processor |
|---|---|---|
| Main advantage | Simpler datapath and smaller area | Higher maximum operating frequency |
| Main disadvantage | Lower maximum clock frequency | Larger area and more design complexity |
| Area result | Lower | Higher |
| Timing result | Meets timing up to 200 MHz in our testing | Meets timing up to 300 MHz in our testing |
| Control complexity | Moderate | High because of hazards and flushing |
| Verification difficulty | Lower | Higher because of overlapping instruction execution |

## 7 Analysis

The biggest lesson from this lab was that pipelining improves performance only when the design also handles hazards correctly. Simply breaking the datapath into stages is not enough by itself. Without forwarding, stalling, and flushing, the processor would execute many programs incorrectly even if the individual stages looked right in isolation.

Compared to the single cycle lab, this one felt much more focused on reasoning about timing between instructions rather than just functionality within one instruction. In the single cycle design, most bugs could be traced to the wrong control signal or datapath path for a specific instruction. In the pipelined design, many of the bugs came from correct pieces interacting incorrectly over multiple cycles.

The synthesis comparison also helped connect the hardware design ideas back to performance tradeoffs. The pipelined processor did exactly what we would expect architecturally: it used

more hardware area, especially sequential area, but it supported a higher frequency. This was a good demonstration that higher performance usually comes from paying more hardware cost and accepting more verification complexity.

The FPGA implementation section became bonus, so we did not complete that part. Even without it, the simulation and synthesis work still gave a strong picture of how the pipelined processor behaves and how it compares against the earlier single cycle version.

# 8 Conclusion

Overall, this lab gave us a much better understanding of what changes when a processor moves from a single cycle design to a pipelined one. We had to take the processor from Lab 2 and reorganize it into stages, preserve instruction correctness across multiple cycles, and add the hazard handling needed to keep the pipeline operating properly.

The ModelSim testing helped us verify the functionality of the pipelined processor, while the Design Compiler results helped us quantify the tradeoff between area and speed. In the end, the pipelined processor behaved as expected: it was more complex and larger than the single cycle version, but it was also able to run at a higher clock frequency.

This lab reinforced the idea that processor design is not just about making instructions work, but also about balancing correctness, timing, hardware cost, and verification effort.

# 9 References

## References


[1] ECEN4243_Lab2.pdf

[2] ECEN4243_Lab3.pdf

[3] Sarah Harris and David Harris, *Digital Design and Computer Architecture, RISC-V Edition*, Elsevier, 2021.

[4] ModelSim was used for simulation and waveform-based verification.

[5] Synopsys Design Compiler was used for ASIC synthesis, timing analysis, and area comparison.

[6] AI was used in the formatting of the Latex document, but was not used to write or edit the content in any way.